RESEARCH SYNTHESIS AND BUILD BLUEPRINT

# Agent Engineering Playbook

One dossier connecting a three-hour masterclass, the Grok Bot Marketplace, and roughly one hundred bookmarked repositories into a single conclusion: a personal engineering agent stack built on skills, verification, and a shippable bot.

## The Lauren Tan Playbook, Reverse-Engineered

Methodology - Skills - Bot - Marketplace

OCTOBER 2026

# Table of Contents

1. Executive Summary 1

2. The Three Sources, Decoded 2

3. Your Bookmark Graph: The Skills Ecosystem 4

4. The Methodology: Trust, Verification, Evals 6

5. The Platform: Grok Bot Marketplace and the Publishing Path 8

6. The Synthesis: What You Are Building 10

7. Build Blueprint: Four Phases to Launch 11

8. Risks, Gaps, and Next Actions 14

# 1. Executive Summary

This dossier answers one question: what, exactly, are you building? It draws on the three sources you supplied - a three-hour Grok Bot masterclass taught by Lauren Tan, the Engineering category of the xAI Grok Bot Marketplace, and her Maven course on how Cursor turned AI agents into better engineers - plus roughly one hundred repositories and links captured across nine screenshots of your bookmarks and chat history. Every item was opened, read, and clustered into themes. The result is not a pile of interesting tools. It is a single, coherent trajectory, and one engineer's fingerprints sit on all three of your anchor links.

That engineer is Lauren Tan: creator of pstack, formerly on the React Compiler core team, Member of Technical Staff at Cursor, and presented as a SpaceX engineer in the masterclass branding - a sign of the SpaceX Super Intelligence rebrand visible in the marketplace footer. Her track record is the reason to pay attention. The Cursor engineering team ran her personal skills ten thousand times in a single week before she open-sourced them, and her workshop claims roughly a thousand agent-merged pull requests per month. Her Maven course teaches the methodology layer, her marketplace bots (tinkabot, Engineering Lead) show the packaging layer, and the masterclass demonstrates the whole system running live.

| ~100 | 10,000x | ~1,000 |
| --- | --- | --- |
| resources analyzed across nine screenshots | weekly uses of Tan's skills at Cursor before open-sourcing | agent-merged PRs per month claimed in her workshop |

Your bookmarks fill in everything the three links leave out. About a quarter of them form the agent skills ecosystem: obra/superpowers, Trail of Bits' skills collection, two skills directories, the SKILL.md open standard, and even an arXiv paper on skill quality. The rest cover MCP servers, autonomous multi-agent systems, memory engines, scraping tools, and fundamentals like LLMs-from-scratch. Read as a graph rather than a list, they supply every layer a published agent product needs: skills, integrations, verification, evaluation, and distribution.

The conclusion, stated once and defended in the chapters that follow: you are reverse-engineering the full pipeline from trustworthy agents to a published marketplace bot. Chapters 2 through 5 present the evidence source by source. Chapter 6 states the thesis precisely and maps each bookmark cluster onto a system layer. Chapter 7 converts the thesis into an eight-week, four-phase build blueprint with a repository shape, a ten-skill starter set, and a publishing checklist. Chapter 8 closes with the risks worth taking seriously and the next five actions to take this week.

# 2. The Three Sources, Decoded

## 2.1 The Masterclass: Trusting Agents Enough to Merge

The YouTube link is a near-three-hour workshop titled 'SpaceX Engineer, Lauren Tan (Masterclass)' on the Get into AI channel, with 10,468 views as of October 22, 2026. Its framing question is the sharpest in the field: how do you trust AI agents enough to let them merge code while you sleep? The video is structured in three chapters - 'How to trust agents enough to let them merge' at 0:00, 'Scaling from one agent to a full team' at 52:27, and 'CI rules for controlling what gets merged' at 1:54:23. That structure is itself the curriculum: verification first, coordination second, automated gating third.

What makes the session valuable is that it is a live setup, not a slide deck. Tan walks through her actual configuration for coordinating agents on real repositories, verifying their output against real artifacts, and enforcing continuous-integration rules that block merges when checks fail. The recurring theme is that autonomy is granted in exchange for demonstrated verification, and that engineering standards - not model cleverness - are what make an agent team safe to scale. It is also a live demonstration of the Grok Bot platform itself, which matters when you choose where to publish.

## 2.2 The Marketplace: Engineering Bots as Products

The second link is the Engineering category of the Grok Bot Marketplace at x.ai. Bots there run on Cursor-hosted cloud computers, users sign in with a Cursor account, and the page footer reads '2026 SpaceX Super Intelligence LLC - Built with Grok.' Featured bots at the time of research included dr eggbot by Lauren Tan, Overheard by Lenny Rachitsky, Tradbot by Claire Vo, and Projects Manager by Eric Zakariasson. The Engineering category is effectively a shelf of finished agent products, each with a narrow job description, and each encoding trust mechanics rather than raw model power. Table 1 summarizes the eight bots listed and the lesson each one teaches.

| Bot | Author | What it does | The lesson it teaches |
|---|---|---|---|
| Projects Manager | Eric Zakariasson | Runs team projects from Notion: one row per project, a channel per project, and tasks that specialist bots claim; it never does specialist work itself. | Orchestration should stay thin; specialists do the work. |
| Researchy | Farzad | A research and fact-check desk that runs every pass on the latest Grok model with live web search, producing sourced, dated claims. | Verification starts with provenance, not confidence. |
| Lingxi's Engineer Bot | Lingxi Li | A hands-off supervisor: boards work, launches cloud agents on the repo you name, watches PRs on a 30-minute cadence, and only asks you to merge. | A supervisor bot is a cadence, not a chatbot. |

| Bot | Author | What it does | The lesson it teaches |
|---|---|---|---|
| tinkabot | Lauren Tan | Wraps an API into a Cursor/Agent plugin (MCP plus skills): data shape first, smallest scaffold that works, prove locally, then publish. | The exact packaging recipe for your own bot. |
| QA bot | Ulysses Ng | Acts as QA on the live deploy: runs the acceptance checklist and says pass or fail before you ship. | Verification is a product, not a step. |
| Engineering Lead | Lauren Tan | Owns the loop: breaks work into Cloud Agents, chases CI, reviews on a weekday cadence, and pings only when something is blocked, ready to merge, or done. | Autonomy is granted after trust, and trust is earned loop by loop. |
| Nightly Audit Engineer | Lingxi Li | Researches a whole codebase overnight, then ships one cleanup PR per area; defaults to 4 a.m. and asks before it starts. | Recurring value beats one-off brilliance. |
| SWE by Cursor | Cursor | The incumbent software engineer bot: writes code, triages issues, monitors production, and more. | The baseline to differentiate from, not to clone. |

Table 1: The eight Engineering bots on the Grok Bot Marketplace and what each teaches

Two observations follow from the table. First, none of these bots sell 'an AI that codes.' They sell a narrow loop with explicit approval gates: a cadence (every 30 minutes, nightly at 4 a.m.), a checklist (acceptance criteria), or a publication rule (nothing sends without your yes). Second, the marketplace rewards verifiable behavior - which is precisely what the methodology in Chapter 4 produces. The marketplace is the distribution layer for verified workflows, and it is where your build eventually lands.

## 2.3 The Course: The Methodology Behind the Bots

The third link is the Maven course 'How Cursor Turned AI Agents Into Better Engineers,' taught by Lauren Tan as part of the Become an AI Native Builder program. Its eleven modules decompose cleanly into four pillars. Pillar one is the Agent Trust Curve: moving from micromanagement to auto-merging PRs, and treating agents like talented but forgetful new hires. Pillar two is verification skills and feature maps: giving the agent a map of reality so it stops guessing. Pillar three is pstack itself - building and maintaining skills that prevent hallucinations, using evals and an eval playbook. Pillar four is operational discipline: refactoring codebases for agents, managing PR sizes, strict CI constraints, the Dune architecture, token usage and ROI, and empowering product teams with Grok Bot.

The course is the connective tissue between the other two sources. The masterclass shows the system running; the marketplace shows it packaged and distributed; the course explains why it works and how to rebuild it for your own team. Taken together they form a complete, teachable pipeline - and every

element of that pipeline has a corresponding cluster in your bookmarks, as the next chapter shows.

# 3. Your Bookmark Graph: The Skills Ecosystem

Nine screenshots capture roughly one hundred saved items: GitHub repositories, browser bookmarks, and chat threads. Clustered by function, six themes emerge, and their proportions are estimated in Figure 1. The largest cluster - about a quarter of everything you saved - is the agent skills ecosystem. The remainder splits across autonomous and multi-agent systems, MCP tooling, the Grok Bot and xAI platform itself, web-data and developer utilities, and fundamentals. The shares matter less than the alignment: every cluster maps onto exactly one layer of the system you are building, which is the argument of Chapter 6.

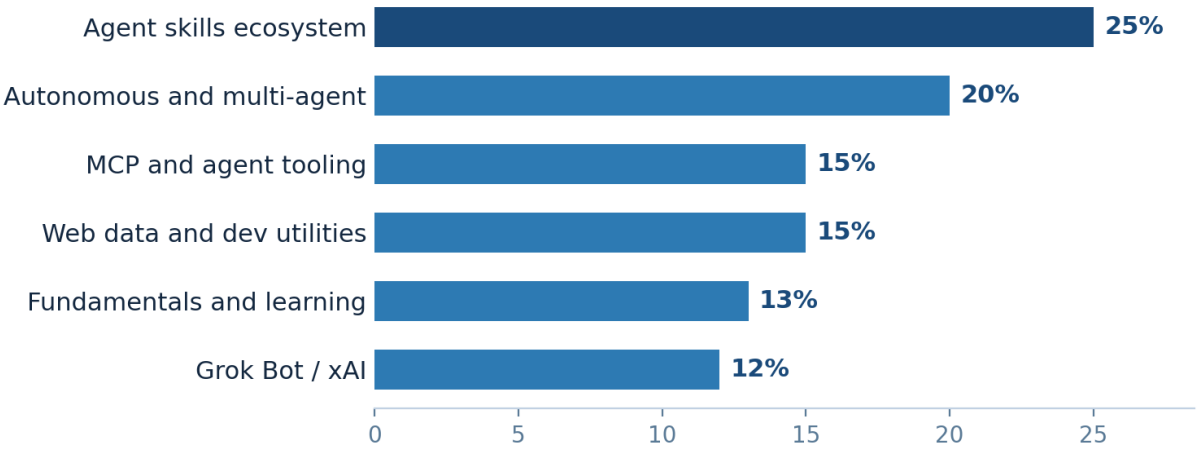



Figure 1: Bookmark clusters by share, ~100 items across nine screenshots

| Cluster | Share | Representative items | Role in the build plan |
|---|---|---|---|
| Agent skills ecosystem | 25% | obra/superpowers, trailofbits/skills, Agent Skills Directory, andrej-karpathy-skills, academic-research-skills, validator-skill, archify, anatomy, Taste Skill, agent-scripts | Format, authoring patterns, and prior art for your own skills pack. |
| Autonomous and multi-agent | 20% | vxcontrol/pentagi, council-of-high-intelligence, omnigent, prime-agent, harvey, strix, supermemory, trycua/cua | Reference architectures for supervision, memory, and computer use. |

| Cluster | Share | Representative items | Role in the build plan |
|---|---|---|---|
| MCP and agent tooling | 15% | serena, idea-reality-mcp, TencentDB-Agent-Memory, pipecat, treg, explain-claude-mod | The integrations your bot will actually call. |
| Grok Bot / xAI | 12% | Marketplace engineering page, Lauren Tan masterclass, tinkabot, xai-org/x-algorithm | The platform, the exemplar bots, and the publishing path. |
| Web data and dev utilities | 15% | Scrapegraph-ai, web-scraper topics, yt-dlp, public-apis, chatwoot, postiz-app, papermark | Utility MCP servers and product analogies for packaging. |
| Fundamentals and learning | 13% | rasbt/LLMs-from-scratch, Python and ML courses, Paul's Online Notes, YC 2026 AI Opportunity Report | Long-game depth; supports everything above. |

Table 2: The six bookmark clusters and the role each plays in your build

## 3.1 The Skills Ecosystem Is Your Center of Gravity

The dominant cluster is not random. obra/superpowers is the most-watched agentic skills framework for Claude Code - a curated plugin of skills, playbooks, and automations with an active community, and its issue tracker is a live record of the ecosystem's growing pains (skill invocation flakiness, per-model relevance debates). trailofbits/skills brings a security engineer's discipline to the same format. Around them you saved domain packs: andrej-karpathy-skills, academic-research-skills, last30days, geo-sleuth, validator-skill, archify, anatomy, Taste Skill, and steipete/agent-scripts. You even bookmarked 'From Anatomy to Smells: An Empirical Study of SKILL.md' - an arXiv paper on what makes skill files good or bad. That is the behavior of someone studying skill ergonomics as a craft, not collecting toys.

## 3.2 The SKILL.md Standard and the Directory Landscape

The skills you collect all speak one format. Per the Agent Skills standard at agentskills.io - originally developed by Anthropic and released as an open standard - a skill is a folder containing a SKILL.md file with metadata (name and description at minimum) and instructions, plus optional scripts/, references/, and assets/ directories. Agents load skills through progressive disclosure: at startup they hold only each skill's name and description (discovery); when a task matches, they read the full instructions (activation); and they then follow them, running bundled code as needed (execution). The context footprint stays small while capability stays deep - which is why the format spread across clients.

Distribution already exists at industrial scale, with sharply different quality models. skills.sh aggregates about 1.37 million skills from public repositories and ranks by install count and recent activity; SkillsMP claims more than two million. skillsdirectory.com takes the opposite bet: roughly 194 thousand skills, every one passing an automated security scan of 120 detection patterns across 10 threat categories, and it

reports that 36 percent of skills in the wild carry security flaws such as prompt injection, credential theft, or data exfiltration. Curated catalogs like AgenticSkills select 193 entries by hand. The implication for you is direct: distribution is table stakes, so quality and safety are the actual differentiators - and both come from the eval and verification discipline in Chapter 4.

### 3.3 MCP and the Autonomous Layer

The remaining clusters are the connective tissue your bot will run on. MCP tooling appears as serena (a powerful code-intelligence toolkit), idea-reality-mcp (pre-build reality checks for ideas), TencentDB-Agent-Memory (agent memory backed by a database), pipecat (voice and media pipelines), and treg (described as OpenRouter for agent tools). The autonomous-systems cluster shows the architectural patterns to borrow: pentagi for fully autonomous agent pipelines, council-of-high-intelligence for structured multi-agent decision-making, omnigent and prime-agent for self-improving loops, harvey for an autonomous sales agent, strix for agentic security testing, supermemory for memory and context, and trycua/cua for computer use. None of these need to be rebuilt; they are the supply chain a skills-first bot wires together.

## 4. The Methodology: Trust, Verification, Evals

### 4.1 The Agent Trust Curve

The Maven course's central model is the Agent Trust Curve: autonomy is granted in exchange for demonstrated verification, and it moves through observable stages. Early on, you micromanage - every diff read line by line. As the agent produces evidence (tests written first, acceptance checklists run, screenshots compared), you shift to reviewing summaries. Eventually, with strict CI constraints blocking any merge that fails checks, the agent merges while you sleep. The masterclass's three chapters trace the same curve from 'how to trust agents enough to let them merge' to 'CI rules for controlling what gets merged.' The curve reframes the human role: not the author of code, but the designer of the evidence that justifies autonomy.

### 4.2 pstack: Anatomy of a Trust System

pstack is the concrete artifact of that methodology: an open-source (MIT, currently v0.15.1) collection of engineering skills, principles, playbooks, subagents, and automations that turns a senior engineer's internal thought process into machinery an agent can execute. The Cursor team used Tan's personal skills ten thousand times in one week before she open-sourced them, and the marketplace listing carries the slogan 'if you want to go fast, go deep first.' Structurally it is a router-plus-specialists system: Poteto Mode receives a task, decides which workflow fits, loads the relevant playbook, and invokes specialized skills. It installs with '/add-plugin pstack', configures model routing with '/setup-pstack', and ships 47 skills, 23 playbooks, 23 principles, and 2 subagents, as shown in Figure 2.

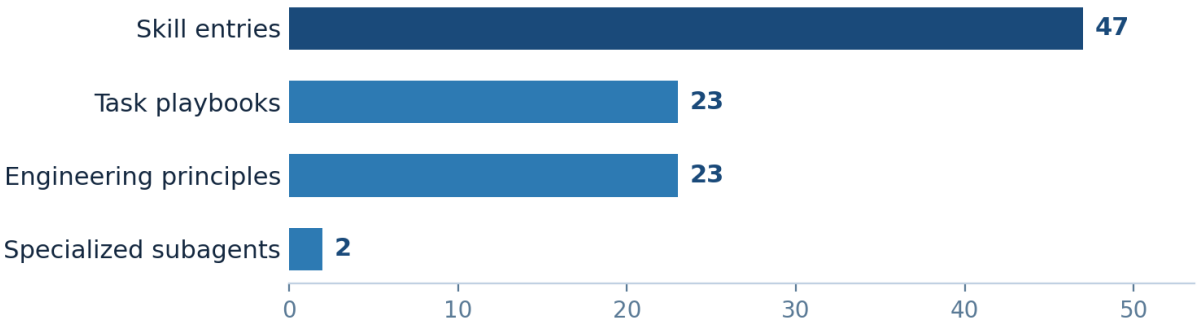



Figure 2: pstack composition per its Cursor marketplace listing (v0.15.1)

| Skill | What it does | Principle it encodes |
|---|---|---|
| /how | Traces how a subsystem actually works. | Build a map of reality before acting. |
| /why | Reconstructs why something was built that way. | Context over guesswork. |
| /recall | Recovers context from previous agent sessions. | Continuity across runs. |
| /architect | Works through types, interfaces, and module structure. | Plan by touching reality. |
| /tdd | Starts from a failing test. | Evidence before implementation. |
| /arena | Produces competing attempts at the same problem. | Selection beats generation. |
| /swarm | Distributes separate slices of a problem across agents. | Parallelism comes after trust. |
| /interrogate | Asks multiple models to attack a completed diff. | Adversarial verification. |
| /create-verification-skill | Gives a project its own mechanism for proving behavior. | Verification as infrastructure. |
| /unslop | Cleans AI-flavored prose from output. | Quality discipline at the seams. |

Table 3: Core pstack skills and the principle each encodes

### 4.3 Verification as Infrastructure

pstack's hallucination defenses are architectural, not prompt tricks. Completed work is verified against real artifacts - the feature exercised in the real product - rather than proxies like 'it compiles' or the agent's self-report. The /interrogate skill subjects a finished diff to multi-model attack; /arena generates competing solutions and selects between them; /create-verification-skill gives every project a bespoke proof harness. Two supporting habits complete the system: 'build the lever' (repeated manual work becomes a tool the agent can invoke, shrinking the error surface) and feature maps (living documents that describe the product's actual surface so the agent works from reality instead of stale memory). This is the layer most collections of agent tools skip, and it is the layer you bookmarked hardest.

### 4.4 Evals, CI, and the Operating Rhythm

The course's operational modules turn trust into routine. Skills are maintained with evals and an eval playbook - they are code, so they rot like code, and each model upgrade demands regression runs. CI is the enforcement arm: strict constraints block merges when checks fail, which is what makes auto-merge safe. Work is structured and PRs are kept small so review stays possible; the Dune architecture and agent-optimized refactoring make codebases navigable by agents; token usage is tracked as ROI, not ignored. The distilled operating rules, quoted from pstack's five lessons, are worth pinning above your desk.

1. Verification is infrastructure.
2. Give the agent a map of reality.
3. Context engineering becomes the human's job.
4. Plan by touching reality.
5. Parallelism comes after trust.

## 5. The Platform: Grok Bot Marketplace and the Publishing Path

Grok Bot is xAI's agent platform: a desktop application for macOS, Windows, and Linux where each bot is created from a job description, receives tasks by text or voice, and runs on Cursor-hosted cloud computers - all bots on an account share one cloud machine, and sign-in uses a Cursor account. The documentation at docs.x.ai/grok-bot covers an approval-centered security model ('Approvals, security, and privacy'), the shared-computer model ('Use the computer and apps'), and enterprise controls such as admin audit logs, with Bot Action Recording as a separate enterprise feature that is off by default. Two behaviors matter most for builders: results are reviewed in one place, and any process worth repeating can be saved as a skill or routine - the platform's own growth loop turns usage into reusable assets.

## 5.1 What the Best Bots Have in Common

Reading Table 1 against the platform mechanics, the successful bots converge on four design choices. Each has a narrow job description instead of general intelligence; each runs on an explicit cadence or trigger rather than waiting to be asked; each emits evidence - a checklist result, a sourced claim, a ready PR - instead of chat; and each routes irreversible actions through an approval gate. None of them require a novel model. They are bundles of workflow, verification, and restraint, which is exactly what the SKILL.md plus MCP plus CI stack from Chapters 3 and 4 assembles. The marketplace is not a display case for model power; it is a shelf for verified loops.

## 5.2 The tinkabot Publishing Recipe

tinkabot (v0.1.0, built with Grok) is the most instructive listing because it is a bot that documents its own construction rules. Its recipe, restated as steps, is the packaging path for your future bot:

1. Prefer the Agent Plugin shape: a root plugin.json plus skills, plus an optional mcp.json - only add rules, hooks, or agents when genuinely required.
2. Name the data shape first; the smallest scaffold that works beats the impressive one that half-works.
3. Keep public, no-auth HTTP MCP servers as zero-dependency stdio servers until an SDK earns its install.
4. Prove the plugin works locally before publishing anything.
5. Ask once about affiliation: if the owner works at the company behind the API, publish to the Cursor Marketplace; otherwise publish to cursor.directory.
6. Never publish to a marketplace unless the owner explicitly says to - the approve gate still blocks the final submit.

tinkabot also reveals the rough edges worth planning around. Grok Bot does not load local plugins from the Cursor local plugin directory, so a plugin proven in the Cursor IDE is not automatically proven on Grok Bot - verification has to be repeated per surface. And the bot itself is a bundle of state: it manages memories, facts, skills, playbooks, and integrations, and it ships pstack inside it. A marketplace bot, in other words, is not a prompt; it is a small operating system for one job, and its quality ceiling is the quality of the skills inside it.

### 5.3 Distribution Reality and Positioning

Distribution spans the official marketplace categories (Engineering, Sales, Marketing, Design, Personal, Recruiting and People, Product, Operations) and the third-party skills directories from Chapter 3. The aggregators index in the millions, which the research site AgenticSkills calls the 'million-skill mirage': totals measure crawl size, not usefulness, and install-count rankings are circular because the leaderboard drives the installs that determine the leaderboard. The winning position is the one your bookmarks already point to - a narrow, outcome-first listing with evidence: security scanning or eval results up front, a README that sells the outcome ('set up complete workspaces in seconds'), not the file format, and a niche defined well enough that the right user recognizes it instantly.

# 6. The Synthesis: What You Are Building

> *You are not collecting repositories. You are reverse-engineering one pipeline: how agents become trustworthy, how skills encode that trust, how a bot packages it, and how a marketplace distributes it.*

Stated as a build order, the thesis has four rungs. The masterclass is the proof at the top: the system works, live, at engineering scale. The Maven course is the methodology that reproduces it: trust curve, verification, evals, CI. The marketplace is the destination: a shelf where verified loops ship as products. And your bookmarks are the supply chain: the skills ecosystem teaches format and craft, MCP provides integrations, the autonomous-systems repos provide architectural patterns, and the fundamentals cluster backs the whole stack with depth. Nothing in the collection is off-path; every cluster funds exactly one layer of the target system.

The target system, defined concretely: a personal engineering agent stack. A skills pack that encodes your standards as SKILL.md folders; MCP wiring into the tools you actually use; a verification harness that produces real-artifact evidence; evals that keep every skill honest across model upgrades; CI gates that make auto-merge safe; and one v0.1 bot published on the marketplace that supervises the loop the way Engineering Lead does - scoped to a domain you know well enough to verify. Table 5 maps the six layers to the bookmarks that feed them and to the first deliverable for each.

| Layer | What it is | Built from (your bookmarks) | First deliverable |
|---|---|---|---|
| 1. Skills pack | SKILL.md folders encoding your engineering standards and repeated workflows. | superpowers, trailofbits/skills, karpathy-skills, academic-research-skills, the SKILL.md anatomy paper | A 10-skill personal pack, each with eval cases. |
| 2. MCP integrations | Real tools the agent can call: code intelligence, memory, data access. | serena, idea-reality-mcp, TencentDB-Agent-Memory, treg, pipecat | Two MCP servers wired and proven locally. |
| 3. Verification harness | Scripts that prove real behavior on real artifacts, per project. | QA bot pattern, validator-skill, pstack's /create-verification-skill | One proof harness attached to your main repo. |
| 4. Evals | Regression tests for the skills themselves, run per model upgrade. | Maven course eval playbook module; superpowers' eval discussions | An eval runner with cases for every skill. |
| 5. CI gates and supervision | Rules that block failing merges; a cadence that watches PRs. | Masterclass CI rules; Engineering Lead and Nightly Audit patterns | A CI workflow plus a supervision cadence document. |
| 6. Publishing | Distribution, positioning, and iteration on real usage. | tinkabot recipe, skills directories, marketplace categories | A v0.1 bot listed publicly with an outcome-first README. |

Table 5: The six layers of your target system, fed by your own bookmarks

Why this reading rather than any other? Because the proportions in Figure 1 are not the proportions of a curious dabbler - they are the proportions of an intentional stack. A quarter invested in skills craft, a fifth in autonomy patterns, steady bets on MCP and the platform, and a fundamentals floor. That distribution matches the six layers almost one to one, and the three anchor links sit on top of it as proof, methodology, and destination. The plan in the next chapter simply makes the stack explicit and puts dates on it.

# 7. Build Blueprint: Four Phases to Launch

The blueprint compresses the pipeline into eight weeks across four phases, with a shippable artifact at every exit. Two ground rules keep it honest. First, install before you build: live inside pstack and superpowers for a week before writing a single skill, because authoring improves immediately once you have felt a mature router-plus-specialists system handle real work. Second, publish in public: the Grok Bot growth playbook is explicit that a configured profile with standing rules, saved routines, and a visible build log compound into distribution - your weekly changelog is part of the product.

| Phase | Weeks | Focus | Concrete actions | Exit criteria |
|---|---|---|---|---|
| 1. Live the methodology | 1-2 | Install before you build. | Install pstack ('/add-plugin pstack') and obra/superpowers; watch the full masterclass; run one real task through Poteto Mode; start a feature map for your main repository. | One week of daily agent work with verification evidence saved. |
| 2. Author your pack | 3-4 | Write 10 SKILL.md skills. | Pick your ten most repeated workflows; author each skill with scripts/ and references/ where useful; write eval cases per skill; run a security self-scan against known patterns. | Ten skills pass their evals; zero critical scan findings. |
| 3. Wire and gate | 5-6 | Integration and trust infrastructure. | Wrap two MCP servers (zero-dep stdio first); build the verification harness for your main repo; add CI constraints that block failing merges; assemble the root plugin.json. | A bot-merged PR lands only after CI and the harness pass. |
| 4. Ship v0.1 | 7-8 | Publish and iterate. | Follow the tinkabot recipe end to end; publish to the marketplace or cursor.directory per the affiliation rule; write the outcome-first README; collect usage; schedule a nightly audit skill. | The bot is listed, with the first external run or install. |

Table 6: The four-phase roadmap with exit criteria

## 7.1 Repository Shape

The repository below is the minimum viable packaging of the six layers from Table 5. It keeps skills platform-agnostic (pure SKILL.md) with the plugin manifest as a thin wrapper, which protects you from platform coupling while keeping every directory installable by standard tooling.

```
agent-stack/
  plugin.json              # root plugin manifest (name, version, author)
  skills/
    repo-onboard/SKILL.md  # /how + /why pattern for new repos
    tdd/SKILL.md           # failing test first
    architect/SKILL.md     # types, interfaces, module structure
    verify-deploy/SKILL.md # acceptance checklist, pass or fail
    nightly-audit/SKILL.md # one cleanup PR per area, on a cadence
    ...                    # 10 skills total in phase 2
  mcp.json                 # optional MCP servers, zero-dep stdio first
  evals/
    cases/*.yaml           # regression cases per skill
    run_evals.py           # pass/fail gate, run on model upgrades
  verification/
    feature-map.md         # living map of the product surface
    harness/               # scripts that prove real behavior
  .github/workflows/ci.yml # CI constraints that block failing merges
```

## 7.2 The Ten-Skill Starter Set

Phase 2's ten skills should come from your own last month of repeated work, but the set below is a proven starting skeleton - each entry maps to an exemplar you already bookmarked, so the pattern to copy is one click away. Author them in this order; the first three pay for the rest.

- /repo-onboard - traces how and why a codebase is shaped the way it is (the /how and /why pattern, and the Researchy habit of citing real files).
- /tdd - starts every change from a failing test; the smallest possible trust unit.
- /architect - works through types, interfaces, and module structure before code exists.
- /verify-deploy - runs the acceptance checklist on a live deploy and says pass or fail (the QA bot pattern).
- /nightly-audit - researches one area of the codebase overnight and ships a single cleanup PR (the Nightly Audit Engineer pattern).
- /pr-sizer - splits work so pull requests stay reviewably small.
- /eval-runner - regresses every skill against its eval cases when the model changes.
- /skill-lint - checks SKILL.md files for anatomy smells and security patterns before publishing.
- /research-brief - produces sourced, dated briefs with live web search (the Researchy pattern).
- /publish-bot - the tinkabot recipe as an executable checklist for future listings.

## 7.3 The Publishing Checklist

| Check | Why it matters |
|---|---|
| Data shape named before code | Prevents building the wrong scaffold and rework at the packaging step. |
| Zero-dependency stdio MCP first | Installs anywhere with no SDK tax; SDKs are earned, not assumed. |
| Proven locally on the target surface | Cursor IDE and Grok Bot behave differently; local proof does not transfer automatically. |
| Evals green on the latest model | Skills rot with model drift; green evals are the renewal date on the trust. |
| Security self-scan against known patterns | One directory reports 36 percent of wild skills carry flaws; scanning is the differentiator. |
| Outcome-first README and description | Directories rank and convert on outcomes, not on file formats. |
| Publish routing decided once | Owner inside the company means the Cursor Marketplace; everyone else means cursor.directory. |
| Approve gate respected | Nothing is ever published without the owner's explicit yes. |

Table 7: Gate every publish through this checklist

After the first publish, the loop, not the launch, is the product. Save every repeated routine as a skill; keep profile rules current; ship the nightly audit on your own repos as the public demonstration of the bot doing its job; and post a short weekly changelog of what the stack merged or caught. Each cycle produces evidence, and evidence is the currency every layer of this pipeline - evals, CI gates, directory rankings, marketplace trust - actually trades in.

# 8. Risks, Gaps, and Next Actions

Every layer of this plan has a known failure mode, and all five are visible inside your own bookmarks - which is the advantage of having read the issue trackers before writing the plan. None are reasons to wait; all are reasons to design the defenses in from day one.

- Platform coupling. tinkabot's own notes flag that Grok Bot cannot load local plugins from the Cursor local directory, so a plugin proven in the IDE is not proven on the bot. Keep skills as pure, platform-agnostic SKILL.md folders and treat plugin manifests as thin, per-platform wrappers that are cheap to rewrite.
- Skill security. The best-scanning directory reports that 36 percent of skills in the wild carry security flaws - prompt injection, credential theft, data exfiltration. Never install an unscanned skill into a bot that has repository or cloud-computer access, and ship your /skill-lint check before your first publish.
- Model drift. The superpowers issue tracker debates whether the framework still adds value on newer models - a healthy argument that proves the point: skills must be re-validated per model version. The eval runner from Phase 2 exists precisely so drift is a scheduled test, not a surprise.

- Discoverability. Aggregators index millions of SKILL.md files and install-count rankings are circular, so volume is not a strategy. Win a narrow niche with evidence - eval results and a security badge up front - and let the outcome-first README do the converting.
- Over-automation before trust. The Trust Curve only advances on verification evidence; granting merge rights in week one is how agent projects die. The phase gates in Table 6 are deliberately conservative about autonomy for this reason.

## 8.1 The Next Five Actions

1. Install pstack and obra/superpowers today, and run one real task through Poteto Mode with the verification evidence saved where you can find it.
2. Watch the masterclass's three chapters against your own repository, and note the exact commit where a CI rule would have blocked a bad merge.
3. Draft your ten-skill list from the workflows you repeated this month, and write the SKILL.md for two of them before the week ends.
4. Stand up the eval runner with three cases per skill and wire it into CI as a job that can fail a build.
5. Open the tinkabot listing and copy its structure line by line into your own plugin.json scaffold, then decide your publish target - Marketplace or cursor.directory - in writing.

The through-line across every link you sent and every repository you saved is a single claim: in agentic engineering, trust is the product. The masterclass shows it being earned in hours, the course shows it being manufactured with evals and CI, the marketplace shows it being sold, and your bookmarks show the supply chain that makes it yours. The stack in this dossier is the smallest complete version of that claim - ten skills, two integrations, one harness, one gate, one listing - and it is buildable in eight weeks with the tools you have already chosen.